KOENIG

step forward



Exception Handling

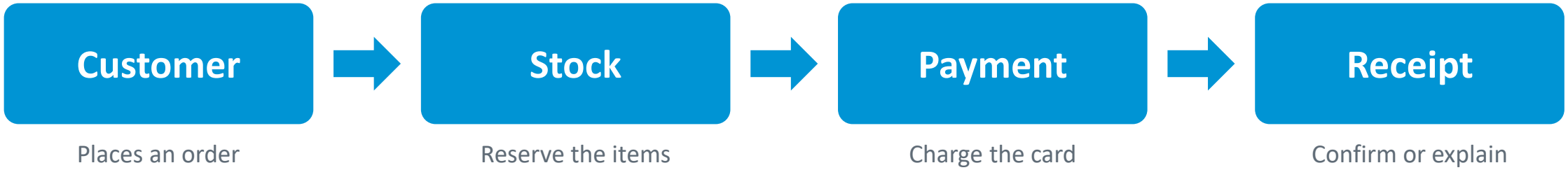
Core Java (Advanced)

Exception Handling

1. Throwable hierarchy
2. Checked vs unchecked
3. try / catch / finally, multi-catch
4. try-with-resources, AutoCloseable
5. Custom business exceptions
6. Anti-patterns to avoid

Demo Scenario: ShopKart Order Placement

Module 3 analysed finished orders. Now we handle what goes wrong while an order is placed. A customer places an order at ShopKart. Stock can run out, a card can be declined, the database can fail. We handle each problem with the right exception.



What can go wrong

- **Out of stock**
- **Card declined**
- **Order missing**
- **Database down**

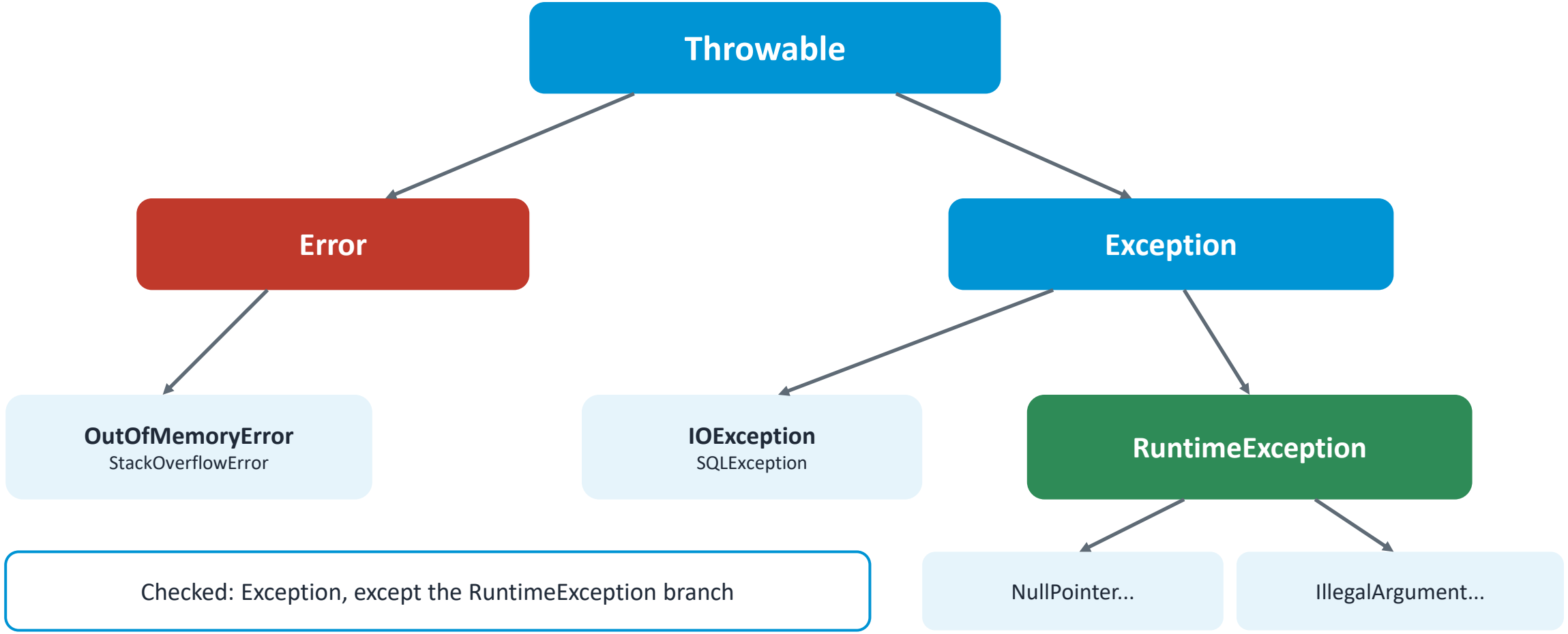
Why Exceptions?

- **Things go wrong:** files missing, cards declined, database down.
- **Return codes** are easy to ignore.
- **An exception** interrupts normal flow and cannot be silently ignored.
- **Stack unwinds** until someone handles it.

```
// Old style: easy to forget the check
int result = reserve("Laptop", 5);
if (result == -1) {      // who remembers?
    // handle
}

// Exception style: cannot be ignored
try {
    inventory.reserve("Laptop", 5);
} catch (InsufficientStockException e) {
    System.out.println(e.getMessage());
}
```

The Throwable Hierarchy



Error, Exception, RuntimeException

Error

Serious JVM problems such as running out of memory. Your code cannot fix them. Do not catch.

Exception

Expected problems like a missing file or a failed database call. These are checked.

RuntimeException

Programming mistakes or rule violations. These are unchecked.

Rule: you handle Exceptions, you never handle Errors

Demo: Why Exceptions and the Hierarchy

What we build

- **Return code:** an ignored -1 from reserve() shows how easily errors are missed.
- **Hierarchy:** walk FileNotFoundException up to Throwable.
- **Unwinding:** a failure three calls deep is handled in main.

How the concept helps

- **Exceptions** cannot be ignored like a return code.
- **Error vs Exception** tells us what we should handle and what we should not.
- **The stack** unwinds until someone catches the problem.

Checked vs Unchecked

Checked

Extends Exception. The compiler FORCES you to catch it or declare throws.

```
void read() throws IOException {
    // callers must handle or declare
}
```

Unchecked

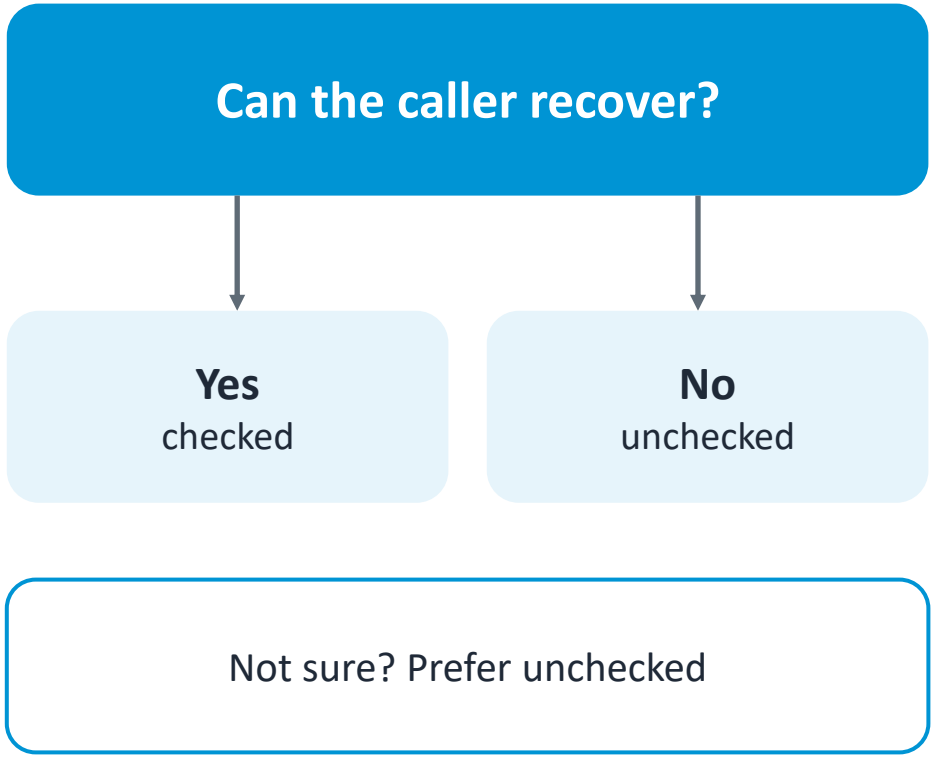
Extends RuntimeException. The compiler does not force anything.

```
void find(int id) {
    throw new OrderNotFoundException(id);
}
```

Checked = recoverable by the caller. Unchecked = bug or central handling

Design Implications

- **Checked:** caller can recover (retry, other card, smaller quantity).
- **Unchecked:** caller cannot help - bug, bad id, broken config.
- **Cost of checked:** 'throws' spreads through every layer.



Demo: Checked vs Unchecked

What we build

- **Checked:** reading a missing receipt file forces us to catch or declare IOException.
- **Unchecked:** OrderNotFoundException needs no throws clause.
- **Declare or handle:** pass the duty up with throws, or catch it here.

How the concept helps

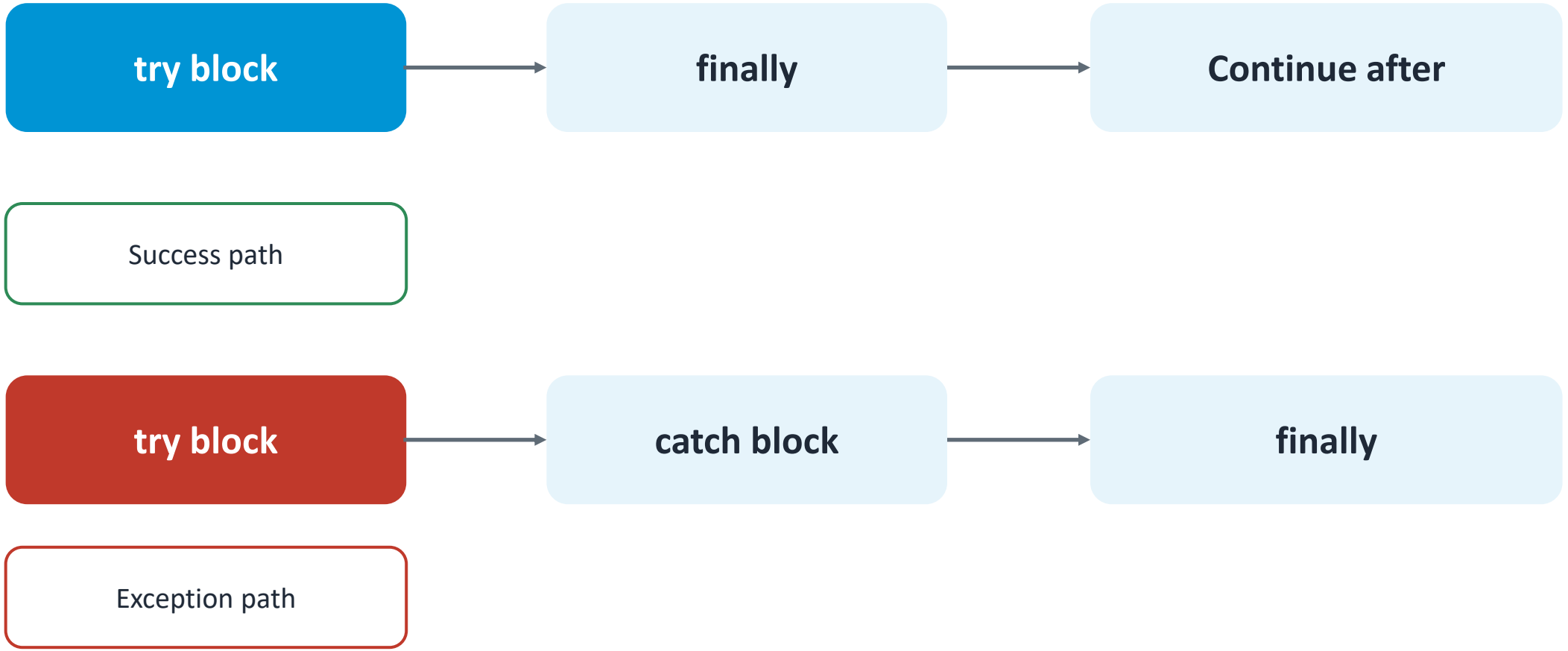
- **Checked** fits problems the caller can recover from.
- **Unchecked** fits bugs and bad input.
- **The compiler** enforces the checked rule for us.

try, catch, finally

- **try** risky code goes here.
- **catch** runs only if that exception type is thrown.
- **finally** always runs - success or failure.
- **Use** finally for cleanup that must happen.

```
try {
    inventory.reserve("Laptop", 5);
    System.out.println("Reserved");
} catch (InsufficientStockException e) {
    System.out.println("Problem: " + e.getMessage());
} finally {
    System.out.println("Always printed");
}
```

How Execution Flows



Lines after the failing line inside try are skipped

Catch Order and Multi-Catch

- **Specific first,** general last.
- **Wrong order** is a compile error.
- **Multi-catch** with | for the same handling.
- **Types in | must** be unrelated.

```
try {
    service.place(1002, "Laptop", 1, 5000);
} catch (PaymentDeclinedException e) {
    askForAnotherCard();
} catch (ShopException e) {
    showGenericError();
}

try {
    parseInput();
} catch (NumberFormatException
         | NullPointerException e) {
    System.out.println("Bad input");
}
```

Demo: try, catch and finally

What we build

- **Three paths:** success, a bad number, and a divide by zero.
- **Catch order:** specific handlers before general ones.
- **Multi-catch:** one handler for NumberFormatException and NullPointerException.

How the concept helps

- **finally** runs on every path, so cleanup is never missed.
- **Specific first** keeps each failure handled by the right code.
- **Multi-catch** removes duplicated handlers.

Custom Exceptions for Business Rules

- **Name the rule:** InsufficientStockException says it all.
- **One base type** for the family: ShopException.
- **Carry data** as fields, not only in the message.
- **Always provide** a message constructor, and a cause one.

```
public class InsufficientStockException
        extends ShopException {

    private final int requested;
    private final int available;

    public InsufficientStockException(
            String product, int requested, int available) {
        super("Not enough " + product);
        this.requested = requested;
        this.available = available;
    }

    public int getShortfall() {
        return requested - available;
    }
}
```

A Business Exception Family




```
try {
    service.place(1002, "Laptop", 1, 5000);
} catch (PaymentDeclinedException e) {   // one member
    askForAnotherCard();
} catch (ShopException e) {              // the whole family
    showGenericError();
}
```

Demo: Custom Business Exceptions

What we build

- **The family:** ShopException with InsufficientStock, PaymentDeclined and OrderNotFound, for a real Customer and Product.
- **Carrying data:** the shortfall is a field, not text in a message.
- **Catching:** one member first, then the whole family.

How the concept helps

- **Named exceptions** say what went wrong in business words.
- **Fields** let the UI show exact numbers.
- **A base class** gives one place to handle every shop problem.

try-with-resources

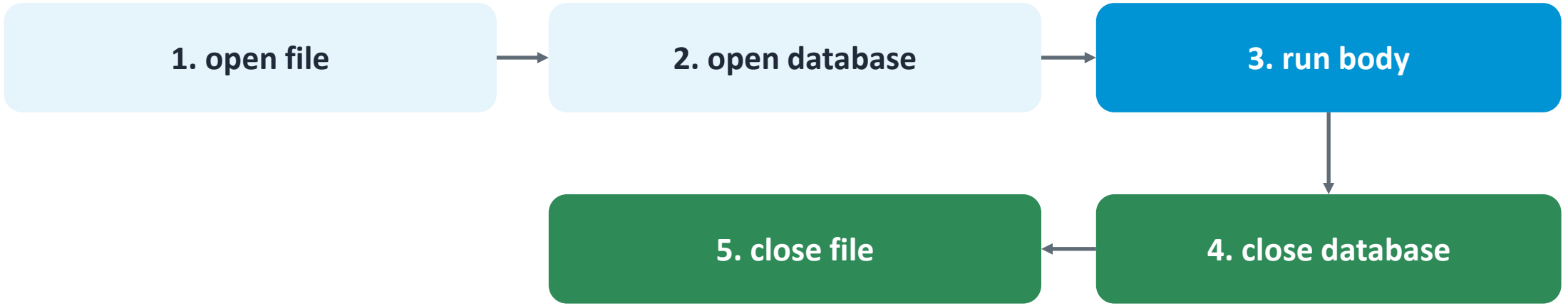
Old way

```
Connection c = null;
try {
    c = open();
    c.use();
} finally {
    if (c != null) {
        c.close();
    }
}
```

try-with-resources

```
try (Connection c = open()) {
    c.use();
}
// closed automatically,
// even if use() fails
```

Resources Close in Reverse Order




```
try (TrackedResource file = new TrackedResource("file");
          TrackedResource db = new TrackedResource("database")) {
    db.use(true);          // fails
}
// close database
// close file   (still closed on failure)
```

The AutoCloseable Contract

- **Interface:** AutoCloseable with one method, close().
- **close() declares** throws Exception - you may narrow it.
- **Good practice:** make close() safe to call twice.
- **Many JDK types** already implement it: readers, streams, connections.

```
public class TrackedResource
        implements AutoCloseable {

    @Override
    public void close() {
        System.out.println("close");
        // release the real resource here
    }
}
```

Demo: try-with-resources

What we build

- **Old way:** open, use, then close in a finally block.
- **try-with-resources:** the same job in one short block.
- **Two resources:** a failure in the body still closes both, in reverse order.

How the concept helps

- **AutoCloseable** makes closing automatic.
- **Reverse order** closes the database before the file it depends on.
- **Leaks** disappear because closing never depends on us remembering.

Exception Anti-Patterns

1. **Swallowing**
   An empty catch hides the problem.

2. **Catching Throwable**
   Catches even JVM Errors.

3. **Control flow**
   Using exceptions as normal logic.

4. **Losing the cause**
   Wrapping without the original.

Anti-Pattern: Swallowing

Don't do this

```
try {
    qty = Integer.parseInt(text);
} catch (Exception e) {
    // ignored
}
```

Do this

```
try {
    qty = Integer.parseInt(text);
} catch (NumberFormatException e) {
    throw new IllegalArgumentException(
        "Quantity must be a number", e);
}
```

Never leave a catch block empty

Anti-Pattern: Catching Throwable

Don't do this

```
try {
    stock.reserve(id);
} catch (Throwable t) {
    return "failed";
}
```

Do this

```
try {
    stock.reserve(id);
} catch (IllegalStateException e) {
    return "failed: " + e.getMessage();
}
```

Catch only what you can actually handle

Anti-Pattern: Control Flow

Don't do this

```
int n = 0;
try {
    while (true) {
        ids[n].length();
        n++;
    }
} catch (ArrayIndexOutOfBoundsException e) {
    // loop ended
}
```

Do this

```
int n = ids.length;

// or simply loop:
for (String id : ids) {
    process(id);
}
```

If you can test it with an if, do not use an exception

Anti-Pattern: Losing the Cause

Don't do this

```
} catch (SQLException e) {
    throw new RuntimeException("Load failed");
}
```

Do this

```
} catch (SQLException e) {
    throw new LoadFailedException(
        "Load failed for " + id, e);
}
```

Wrap with context AND the original cause

Demo: Anti-Patterns and Their Fixes

What we build

- **Swallowing:** a silent 0 versus a clear IllegalArgumentException.
- **Catching Throwable:** hides even real JVM errors.
- **Control flow and lost causes:** length instead of exceptions, and the cause kept.

How the concept helps

- **Specific catches** keep real problems visible.
- **An if or a loop** beats an exception for normal logic.
- **Context plus cause** makes a failure easy to diagnose.

Best-Practice Checklist

1. Name exceptions after the business rule
2. Catch specific types, not Throwable
3. Use try-with-resources for AutoCloseable
4. Pass the cause when wrapping
5. Never leave a catch block empty
6. Fail with details: fields, ids, values

Key Takeaways

- **Hierarchy:** Throwable splits into Error and Exception.
- **Checked vs unchecked:** choose by whether the caller can recover.
- **try-with-resources:** closes in reverse order, even on failure.
- **Custom exceptions:** name the rule and carry data.
- **Avoid:** swallowing, Throwable, exceptions as control flow.



Happy Learning :)